

PROGRAMMING IN HASKELL



Capítulo 4 - Definindo Funções

Expressões Condicionais

Funções podem ser definidas através de expressões condicionais.

```
abs :: Int → Int  
abs n = if n >= 0 then n else -n
```

abs toma um inteiro **n** e retorna **n** se ele for não-negativo e **-n** caso contrário.

Expressões condicionais podem ser aninhadas:

```
signum :: Int → Int
signum n = if n < 0 then -1 else
            if n == 0 then 0 else 1
```

Nota:

- Em Haskell, expressões condicionais devem sempre conter um *else*, para que se evite problemas de ambiguidade com condicionais aninhados.

Equações Guardadas

Uma alternativa para expressões condicionais, são as equações guardadas.

```
abs n | n >= 0      = n  
      | otherwise = -n
```

Equações guardadas podem ser utilizadas para facilitar a compreensão de definições que envolvem múltiplas condições:

```
signum n | n < 0      = -1
         | n == 0     = 0
         | otherwise = 1
```

Nota:

- A condição otherwise está definida no prelude como `otherwise = True`.

Casamento de Padrões

Muitas funções são mais facilmente definidas através do casamento de padrões (pattern matching) sobre seus argumentos.

```
not :: Bool → Bool  
not False = True  
not True  = False
```

`not` mapeia `False` em `True`, e `True` em `False`.

Funções podem ser definidas de várias formas usando casamento de padrões. Por exemplo

```
(&&) :: Bool → Bool → Bool
True  && True   = True
True  && False  = False
False && True   = False
False && False  = False
```

pode ser definido de forma mais compacta por

```
True && True = True
_    && _    = False
```

No entanto, a definição a seguir é mais inteligente, pois ela evita que o segundo argumento da função seja avaliado caso o primeiro argumento tenha sido avaliado como False:

```
True  && b = b  
False && _ = False
```

Nota:

- O símbolo `_` é um padrão curinga que pode ser casado com qualquer valor de argumento.

- Padrões são verificados em ordem. Por exemplo, a seguinte definição sempre retorna False:

```
_      && _      = False
True && True = True
```

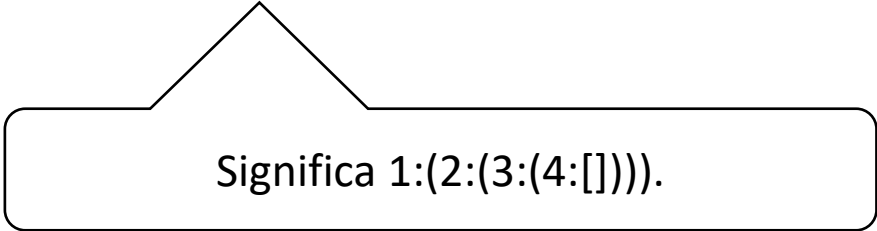
- Padrões não devem repetir variáveis. Por exemplo, a seguinte definição é um erro:

```
b && b = b
_ && _ = False
```

Padrões sob Listas

Internamente, cada lista não-vazia é construída através do uso repetido do operador (:) chamado “cons” que adiciona um elemento ao início (cabeça) da lista.

[1, 2, 3, 4]



Significa 1:(2:(3:(4:[]))).

Funções que operam sobre listas podem ser definidas através do padrão x:xs.

```
head :: [a] → a  
head (x:_) = x
```

```
tail :: [a] → [a]  
tail (_:xs) = xs
```

head e **tail** mapeiam qualquer lista não-vazia para o primeiro e restantes elementos.

Nota:

- Padrões $x:xs$ serão casados apenas a listas não-vazias:

```
> head []  
*** Exception: empty list
```

- Padrões $x:xs$ devem estar entre parênteses, pois a aplicação de funções tem prioridade sobre $(:)$. Por exemplo, a seguinte definição é um erro:

```
head x:_ = x
```

Expressões Lambda

Funções podem ser construídas sem que as mesmas sejam nomeadas através de expressões lambda.

$\lambda x \rightarrow x + x$

$\backslash x \rightarrow x + x$

A função anônima que toma um número x e retorna $x + x$ como resultado.

Nota:

- O símbolo λ é a letra grega lambda, e é representada em Haskell pelo símbolo $\backslash$.
- Na matemática, funções anônimas são usualmente denotadas pelo símbolo $\mapsto$, tal como $x \mapsto x + x$.
- Em Haskell, o uso do símbolo λ para funções anônimas se origina no cálculo lambda, a teoria sobre funções na qual o Haskell se baseia.

Qual a Utilidade?

Expressões lambda podem ser usadas para dar sentido formal a funções currificadas.

Por exemplo:

`add x y = x + y`

significa

`add =  $\lambda x \rightarrow (\lambda y \rightarrow x + y)$`

Expressões lambda podem ser usadas para evitar nomear funções que são referenciadas uma única vez.

Por exemplo:

```
odds n = map f [0..n-1]
      where
        f x = x*2 + 1
```

Pode ser simplificado em

```
odds n = map ( $\lambda x \rightarrow x*2 + 1$ ) [0..n-1]
```

Seções de Operadores

Um operador binário que é usado entre seus dois argumentos pode ser convertido em uma função currificada para ser usada antes de seus dois argumentos através do uso de parênteses.

Por exemplo:

```
> 1+2
```

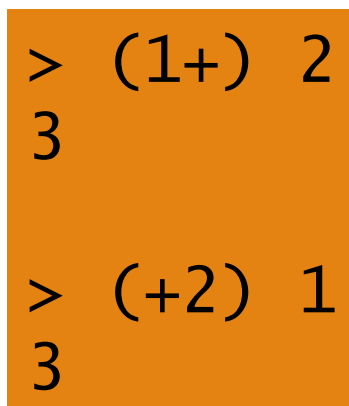
```
3
```

```
> (+) 1 2
```

```
3
```

Esta convenção permite que um dos argumentos seja incluído nos parênteses.

Por exemplo:



The image shows two examples of sections in a lambda calculus notation, highlighted in an orange box. The first example is $> (1+) 2$ with a 3 below the $>$. The second example is $> (+2) 1$ with a 3 below the $>$.

De forma geral, se $\oplus$ é um operador, então funções na forma $(\oplus)$, $(x\oplus)$ e $(\oplus y)$ são chamadas seções.

Qual a Utilidade?

Seções podem ser usada na construção de algumas funções úteis. Por exemplo:

$(1+)$

- Função sucessora

$(1/)$

- Função do inverso multiplicativo

$(*2)$

- Função do dobro

$(/2)$

- Função da biseção

Exercícios

- (1) Considere a função safetail que tem o mesmo comportamento que tail, exceto que *safetail* mapeia a lista vazia para a lista vazia, enquanto *tail* acusa um erro. Defina *safetail* usando:
- (a) uma expressão condicional;
 - (b) equações guardadas;
 - (c) casamento de padrões.

Dica: a função `null :: [a] → Bool` pode ser usada para testar se uma lista é vazia.

(2) Dê três definições para o operador lógico ou (`||`) usando casamento de padrões.

(3) Redefina a seguinte versão de (`&&`) usando condicionais ao invés de padrões:

```
True && True = True
_      && _    = False
```

(4) Faça o mesmo para:

```
True && b = b
False && _ = False
```